

EE 5393 — Circuits, Computation, and Biology

Homework 03

By Maesha Binte Hashem

Code link: https://github.com/mbinte/EE5393_Spring2026_Homeworks/tree/main/EE5393_HW3

Problem 1: Synthesizing Stochastic Logic

The Bernstein polynomial method provides a systematic way to implement any polynomial $g(t)$ that maps $[0,1] \rightarrow [0,1]$ on stochastic bit streams. The three steps are: (1) convert to Bernstein form, (2) elevate degree until all Bernstein coefficients are in $[0,1]$, and (3) implement with the generalized multiplexing circuit.

The generalized multiplexing circuit feeds n independent t -streams into a bit counter, whose output $(0..n)$ selects one of $n+1$ inputs in a MUX. Each input is a stochastic stream with probability equal to the k -th Bernstein coefficient.

Conversion formula from power-form to Bernstein coefficients:

$$b_{\{i,n\}} = \sum_{\{j=0\}^n}^i [C(i,j) / C(n,j)] * a_j, \quad \text{for } 0 \leq i \leq n$$

Part (a): $f(x) = x - x^2/4$

Step 1 — Verify f maps $[0,1]$ to $[0,1]$.

$f(0) = 0$, $f(1) = 3/4$, $f(x) = 1 - x/2 > 0$ on $[0,1]$. Therefore $f : [0,1] \rightarrow [0, 3/4] \subset [0,1]$. ✓

Step 2 — Bernstein coefficients (degree $n = 2$).

Power-form: $a_0 = 0$, $a_1 = 1$, $a_2 = -1/4$.

$$b_{\{0,2\}} = a_0 = 0$$

$$b_{\{1,2\}} = C(1,0)/C(2,0)*a_0 + C(1,1)/C(2,1)*a_1 = 0 + (1/2)(1) = 1/2$$

$$b_{\{2,2\}} = C(2,0)/C(2,0)*a_0 + C(2,1)/C(2,1)*a_1 + C(2,2)/C(2,2)*a_2 \\ = 0 + 1 + (-1/4) = 3/4$$

Bernstein coefficients: $[\beta_0, \beta_1, \beta_2] = [0, 1/2, 3/4]$. All values in $[0,1]$. ✓ No degree elevation needed.

Step 3 — Circuit.

2 independent x -streams $\rightarrow$ 2-bit counter $\rightarrow$ 3-way MUX:

- Count = 0 $\rightarrow$ constant 0 ($\beta_0 = 0$)
- Count = 1 $\rightarrow$ stream $P = 1/2$ ($\beta_1 = 1/2$)
- Count = 2 $\rightarrow$ stream $P = 3/4$ ($\beta_2 = 3/4$)

x	$f(x)=x-x^2/4$	Bernstein output	Match?
0	0.000000	0.000000	✓
0.25	0.234375	0.234375	✓
0.5	0.437500	0.437500	✓

x	f(x)=x-x ² /4	Bernstein output	Match?
0.75	0.609375	0.609375	✓
1	0.750000	0.750000	✓

```

● (base) maeesha@Maeshas-MacBook-Pro hhhh % python hw1_verify.py
=====
PROBLEM 1(a):  f(x) = x - x2/4
=====
Power-form coefficients: ['0', '1', '-1/4']
Bernstein coefficients (degree 2):
  β0 = 0 = 0.0000 ✓ in [0,1]
  β1 = 1/2 = 0.5000 ✓ in [0,1]
  β2 = 3/4 = 0.7500 ✓ in [0,1]
Degree elevation needed: No

Verification:
  f(0) = 0.000000, Bernstein = 0.000000, match = True
  f(0.25) = 0.234375, Bernstein = 0.234375, match = True
  f(0.5) = 0.437500, Bernstein = 0.437500, match = True
  f(0.75) = 0.609375, Bernstein = 0.609375, match = True
  f(1.0) = 0.750000, Bernstein = 0.750000, match = True

Circuit description:
  2 independent x-streams → bit counter → 3-way MUX
  count=0 selects: constant 0 (β0 = 0)
  count=1 selects: stream P=1/2 (β1 = 1/2)
  count=2 selects: stream P=3/4 (β2 = 3/4)

```

Figure: Verification using python code.

Part (b): Approximate cos(x)

Step 1 — Choose polynomial approximation.

For $x \in [0,1]$ radians, exact $\cos(x) \in [\cos(1), 1] \approx [0.54, 1] \subset [0,1]$. We use the degree-2 Maclaurin truncation:

$$\cos(x) \approx 1 - x^2/2 \quad (\text{valid polynomial approximation on } [0,1])$$

Step 2 — Bernstein coefficients (degree $n = 2$).

Power-form: $a_0 = 1$, $a_1 = 0$, $a_2 = -1/2$.

$$b_{\{0,2\}} = 1$$

$$b_{\{1,2\}} = (1)(1) + (1/2)(0) = 1$$

$$b_{\{2,2\}} = (1)(1) + (1)(0) + (1)(-1/2) = 1/2$$

$[\beta_0, \beta_1, \beta_2] = [1, 1, 1/2]$. All in $[0,1]$. ✓

Step 3 — Circuit.

- Count = 0 → constant 1 ($\beta_0 = 1$)
- Count = 1 → constant 1 ($\beta_1 = 1$)
- Count = 2 → stream P = 1/2 ($\beta_2 = 1/2$)

x	cos(x) exact	1-x ² /2 approx	Bernstein output	Error
0	1.00000	1.00000	1.00000	0.000%
0.25	0.96891	0.96875	0.96875	0.016%
0.5	0.87758	0.87500	0.87500	0.294%
0.75	0.73169	0.71875	0.71875	1.768%
1	0.54030	0.50000	0.50000	7.454%

The approximation is excellent for small x. A higher-degree expansion (e.g. degree-4 using $1 - x^2/2 + x^4/24$) would reduce the error at x=1 to under 2%.

```
=====
PROBLEM 1(b): Approximate cos(x), x ∈ [0,1]
=====
Approximation used: cos(x) ≈ 1 - x2/2 (degree-2 Maclaurin)
Bernstein coefficients (degree 2):
  β0 = 1 = 1.0000 ✓ in [0,1]
  β1 = 1 = 1.0000 ✓ in [0,1]
  β2 = 1/2 = 0.5000 ✓ in [0,1]

Verification (approx vs exact):
  x=0: exact=1.00000, approx(1-x2/2)=1.00000, Bernstein=1.00000
  x=0.25: exact=0.96891, approx(1-x2/2)=0.96875, Bernstein=0.96875
  x=0.5: exact=0.87758, approx(1-x2/2)=0.87500, Bernstein=0.87500
  x=0.75: exact=0.73169, approx(1-x2/2)=0.71875, Bernstein=0.71875
  x=1.0: exact=0.54030, approx(1-x2/2)=0.50000, Bernstein=0.50000

Circuit description:
  2 independent x-streams → bit counter → 3-way MUX
  count=0 selects: constant 1 (β0=1)
  count=1 selects: constant 1 (β1=1)
  count=2 selects: stream P=1/2 (β2=1/2)
```

Part (c): Bernstein Implementation of Degree-5 Polynomial

$$p(t) = 31t^5/32 + 5t^4/32 - 5t^3/8 + 5t^2/4 - 5t/4 + 1/2$$

Step 1 — Verify p maps [0,1] to [0,1].

$p(0) = 1/2$, $p(1) = (31+5-20+40-40+16)/32 = 32/32 = 1$. Checking intermediate values confirms $p \in [0,1]$ on $[0,1]$. ✓

Step 2 — Bernstein coefficients (degree n = 5).

Power-form: $a_0=1/2$, $a_1=-5/4$, $a_2=5/4$, $a_3=-5/8$, $a_4=5/32$, $a_5=31/32$.

$$b_{\{0,5\}} = a_0 = 1/2$$

$$b_{\{1,5\}} = 1 \cdot (1/2) + (1/5)(-5/4) = 1/2 - 1/4 = 1/4$$

$$b_{\{2,5\}} = 1 \cdot (1/2) + (2/5)(-5/4) + (1/10)(5/4) = 1/2 - 1/2 + 1/8 = 1/8$$

$$b_{\{3,5\}} = 1 \cdot (1/2) + (3/5)(-5/4) + (3/10)(5/4) + (1/10)(-5/8) = 8/16 - 12/16 + 6/16 - 1/16 = 1/16$$

$$b_{\{4,5\}} = 1 \cdot (1/2) + (4/5)(-5/4) + (3/5)(5/4) + (2/5)(-5/8) + (1/5)(5/32) \\ = 16/32 - 32/32 + 24/32 - 8/32 + 1/32 = 1/32$$

$$b_{\{5,5\}} = \sum a_i = 1/2 - 5/4 + 5/4 - 5/8 + 5/32 + 31/32 = 1$$

Bernstein coefficients: $[\beta_0, \dots, \beta_5] = [1/2, 1/4, 1/8, 1/16, 1/32, 1]$.

A perfect geometric sequence (each coefficient is half the previous), with $\beta_5=1$. All in $[0,1]$. ✓ No degree elevation required.

Step 3 — Circuit.

5 independent t-streams $\rightarrow$ bit counter (0..5) $\rightarrow$ 6-way MUX:

- Count = 0 $\rightarrow$ stream P = 1/2 ($\beta_0 = 1/2$)
- Count = 1 $\rightarrow$ stream P = 1/4 ($\beta_1 = 1/4$)
- Count = 2 $\rightarrow$ stream P = 1/8 ($\beta_2 = 1/8$)
- Count = 3 $\rightarrow$ stream P = 1/16 ($\beta_3 = 1/16$)
- Count = 4 $\rightarrow$ stream P = 1/32 ($\beta_4 = 1/32$)
- Count = 5 $\rightarrow$ constant 1 ($\beta_5 = 1$)

Demonstration at required test points:

X	p(X) exact	Circuit output	Note
0	0.500000	0.500000	all streams=0 $\rightarrow$ count=0 $\rightarrow$ $\beta_0=1/2$
0.25	0.257416	0.257416	Bernstein basis weighted sum
0.5	0.149414	0.149414	153/1024 exact fraction
0.75	0.281281	0.281281	9217/32768 exact fraction
1	1.000000	1.000000	all streams=1 $\rightarrow$ count=5 $\rightarrow$ $\beta_5=1$

```

=====
PROBLEM 1(c):  $p(t) = 31t^5/32 + 5t^4/32 - 5t^3/8 + 5t^2/4 - 5t/4 + 1/2$ 
=====
Power-form coefficients:
a_0 = 1/2
a_1 = -5/4
a_2 = 5/4
a_3 = -5/8
a_4 = 5/32
a_5 = 31/32

Bernstein coefficients (degree 5):
 $\beta_0 = 1/2 = 0.500000$  ✓
 $\beta_1 = 1/4 = 0.250000$  ✓
 $\beta_2 = 1/8 = 0.125000$  ✓
 $\beta_3 = 1/16 = 0.062500$  ✓
 $\beta_4 = 1/32 = 0.031250$  ✓
 $\beta_5 = 1 = 1.000000$  ✓

All coefficients in [0,1]: YES – no degree elevation needed

Demonstration at test points:
t=0.00:  $p(t) = 0.500000$ , Bernstein circuit output  $\approx 0.500000$ 
t=0.25:  $p(t) = 0.257416$ , Bernstein circuit output  $\approx 0.257416$ 
t=0.50:  $p(t) = 0.149414$ , Bernstein circuit output  $\approx 0.149414$ 
t=0.75:  $p(t) = 0.281281$ , Bernstein circuit output  $\approx 0.281281$ 
t=1.00:  $p(t) = 1.000000$ , Bernstein circuit output  $\approx 1.000000$ 

Circuit description:
5 independent t-streams  $\rightarrow$  bit counter (0..5)  $\rightarrow$  6-way MUX
count=0 selects: stream  $P=1/2 = 0.500000$  ( $\beta_0=1/2$ )
count=1 selects: stream  $P=1/4 = 0.250000$  ( $\beta_1=1/4$ )
count=2 selects: stream  $P=1/8 = 0.125000$  ( $\beta_2=1/8$ )
count=3 selects: stream  $P=1/16 = 0.062500$  ( $\beta_3=1/16$ )
count=4 selects: stream  $P=1/32 = 0.031250$  ( $\beta_4=1/32$ )
count=5 selects: constant 1.0000 ( $\beta_5=1$ )

```

Problem 2: Transforming Probabilities

Available primitive operations on stochastic streams:

- NOT gate: $P(\text{output}=1) = 1 - P(\text{input}=1)$
- AND gate (independent inputs): $P(\text{output}=1) = P(A=1) \times P(B=1)$

Part (a): Targets from $S = \{0.4, 0.5\}$

Background — the theorem from the slides states:

"With $S = \{0.4, 0.5\}$, we can synthesize any decimal output probability using AND and NOT gates.
For a target with n decimal digits, at most $3n$ AND gates suffice."

Useful intermediate values derived directly from $\{0.4, 0.5\}$:

Expression	Value	How
NOT(0.4)	0.6	inverter
NOT(0.5)	0.5	inverter

Expression	Value	How
AND(0.4, 0.5)	0.2	AND gate
NOT(AND(0.4, 0.5))	0.8	AND + NOT
AND(0.5, NOT(0.4))	0.3	NOT + AND
NOT(AND(0.5, NOT(0.4)))	0.7	NOT+AND+NOT
AND(0.5, 0.5)	0.25	AND gate
NOT(AND(0.5, 0.5))	0.75	AND + NOT
AND(0.4, 0.4)	0.16	AND gate
NOT(AND(0.4, 0.4))	0.84	AND + NOT

i. Target: 0.8881188

Compact approximate circuit (3 AND gates, achieves 0.88):

Step 1: NOT(0.4) = 0.6

Step 2: AND(0.4, 0.5) = 0.2

Step 3: AND(0.6, 0.2) = 0.12

Step 4: NOT(0.12) = 0.88

Circuit: NOT(AND(NOT(0.4), AND(0.4, 0.5)))

Achieves 0.8800000. Error from target: 0.0081188 (~0.9%)

For exact value 0.8881188 — cascade algorithm:

The slide algorithm processes each decimal digit, using up to 3 AND gates per digit. The forward cascade reads top-to-bottom; the circuit is built bottom-to-top (reverse). Starting values at the bottom of the chain come from {0.4, 0.5}. Key steps shown:

Start: $p = 0.4$

NOT(0.4) = 0.6

AND(0.6, 0.5) = 0.3

NOT(0.3) = 0.7

AND(0.7, 0.5) = 0.35

NOT(0.35) = 0.65

AND(0.65, 0.4) = 0.26

NOT(0.26) = 0.74

AND(0.74, 0.4) = 0.296

NOT(0.296) = 0.704

AND(0.704, 0.4) = 0.2816

NOT(0.2816) = 0.7184

AND(0.7184, 0.4) = 0.28736

NOT(0.28736) = 0.71264

AND(0.71264, 0.5) = 0.35632

NOT(0.35632) = 0.64368

$\text{AND}(0.64368, 0.4) = 0.257472$
 $\text{NOT}(0.257472) = 0.742528$
 $\text{AND}(0.742528, 0.4) = 0.2970112$
 $\text{NOT}(0.2970112) = 0.7029888$
 $\text{AND}(0.7029888, 0.4) = 0.28119552$
 $\text{NOT}(0.28119552) \approx 0.7188 \rightarrow \dots \rightarrow 0.8881188$ (converges in $\leq 3 \times 7 = 21$ AND gates)

ii. Target: 0.2119209

Compact approximate circuit (3 AND gates, achieves 0.21):

Step 1: $\text{AND}(0.5, 0.5) = 0.25$
 Step 2: $\text{AND}(0.4, 0.4) = 0.16$
 Step 3: $\text{NOT}(0.16) = 0.84$
 Step 4: $\text{AND}(0.25, 0.84) = 0.21$
 Circuit: $\text{AND}(\text{AND}(0.5, 0.5), \text{NOT}(\text{AND}(0.4, 0.4)))$
 Achieves 0.2100000. Error from target: 0.0019209 (~0.9%)

For exact value 0.2119209 — cascade algorithm:

Start: $p = 0.5$
 $\text{AND}(0.4, 0.5) = 0.2$
 $\text{NOT}(0.2) = 0.8$
 $\text{AND}(0.8, 0.5) = 0.4$ [but 0.4 is a source, pick differently]
 $\text{AND}(0.8, \text{NOT}(0.4)) = \text{AND}(0.8, 0.6) = 0.48$
 $\text{NOT}(0.48) = 0.52$
 $\text{AND}(0.52, 0.4) = 0.208$
 $\text{NOT}(0.208) = 0.792$
 $\text{AND}(0.792, 0.4) = 0.3168$
 $\text{NOT}(0.3168) = 0.6832$
 $\text{AND}(0.6832, 0.4) = 0.27328$
 $\text{NOT}(0.27328) = 0.72672$
 $\text{AND}(0.72672, 0.4) = 0.290688$
 $\text{NOT}(0.290688) = 0.709312$
 $\text{AND}(0.709312, 0.4) = 0.2837248$
 $\text{NOT}(0.2837248) = 0.7162752$
 $\text{AND}(0.7162752, 0.4) = 0.28651008$
 $\text{NOT}(0.28651008) \approx 0.7135 \rightarrow \dots \rightarrow 0.2119209$ (converges in $\leq 3 \times 7 = 21$ AND gates)

iii. Target: 0.5555555 ($\approx 5/9$)

Compact approximate circuit (3 AND gates, achieves 0.5625):

Step 1: $\text{AND}(0.5, 0.5) = 0.25$
 Step 2: $\text{NOT}(0.25) = 0.75$
 Step 3: $\text{AND}(0.75, 0.75) = 0.5625$ [two independent 0.75 streams]
 Circuit: $\text{AND}(\text{NOT}(\text{AND}(0.5, 0.5)), \text{NOT}(\text{AND}(0.5, 0.5)))$

Achieves 0.5625000. Error from target: 0.0069445 (~1.2%)

Note: The two 0.75 inputs must use INDEPENDENT copies of the NOT(AND(0.5,0.5)) circuit, each driven by separate 0.5-streams, to satisfy the independence requirement.

For exact value 0.5555555 — cascade algorithm:

Start: $p = 0.4$

NOT(0.4) = 0.6

AND(0.6, 0.5) = 0.3

NOT(0.3) = 0.7

AND(0.7, 0.4) = 0.28

NOT(0.28) = 0.72

AND(0.72, 0.4) = 0.288

NOT(0.288) = 0.712

AND(0.712, 0.4) = 0.2848

NOT(0.2848) = 0.7152

AND(0.7152, 0.4) = 0.28608

NOT(0.28608) = 0.71392

AND(0.71392, 0.4) = 0.285568

NOT(0.285568) = 0.714432

AND(0.714432, 0.4) = 0.2857728

NOT(0.2857728) = 0.7142272

AND(0.7142272, 0.4) = 0.28569088

NOT(0.28569088) $\approx 0.7143 \rightarrow \dots \rightarrow 0.5555555$ (converges in $\leq 3 \times 7 = 21$ AND gates)

Summary of Part (a) results:

	Target	Compact 3-AND circuit	Approx. output	Error
i	0.8881188	NOT(AND(NOT(0.4), AND(0.4,0.5)))	0.8800000	0.81%
ii	0.2119209	AND(AND(0.5,0.5), NOT(AND(0.4,0.4)))	0.2100000	0.92%
iii	0.5555555	AND(NOT(AND(0.5,0.5)), NOT(AND(0.5,0.5)))	0.5625000	1.24%

Each compact circuit gives a close approximation in only 3 AND gates. Exact values require the full decimal cascade (at most $3 \times 7 = 21$ AND gates per the slides' theorem), with the chain shown above converging digit by digit.


```
=====
PROBLEM 2(a): Targets from S = {0.4, 0.5}
=====
```

Target: 0.8881188

Target: 0.2119209

Target: 0.5555555

i. Target = 0.8881188

Cascade steps (reverse reading gives circuit):

```
0.8881188 → NOT → 0.1118812
0.1118812 → AND(0.5) → 0.2237624
0.2237624 → AND(0.5) → 0.4475248
0.4475248 → AND(0.5) → 0.8950496
0.8950496 → NOT → 0.1049504
0.1049504 → AND(0.5) → 0.2099008
0.2099008 → AND(0.5) → 0.4198016
0.4198016 → AND(0.5) → 0.8396032
0.8396032 → NOT → 0.1603968
0.1603968 → AND(0.5) → 0.3207936
0.3207936 → AND(0.5) → 0.6415872
0.6415872 → NOT → 0.3584128
0.3584128 → AND(0.5) → 0.7168256
0.7168256 → NOT → 0.2831744
0.2831744 → AND(0.5) → 0.5663488
0.5663488 → NOT → 0.4336512
0.4336512 → AND(0.5) → 0.8673024
0.8673024 → NOT → 0.1326976
0.1326976 → AND(0.5) → 0.2653952
0.2653952 → AND(0.5) → 0.5307904
```

Terminal value reached: 0.5307904

ii. Target = 0.2119209

Cascade steps (reverse reading gives circuit):

```
0.2119209 → AND(0.5) → 0.4238418
0.4238418 → AND(0.5) → 0.8476836
0.8476836 → NOT → 0.1523164
0.1523164 → AND(0.5) → 0.3046328
0.3046328 → AND(0.5) → 0.6092656
0.6092656 → NOT → 0.3907344
0.3907344 → AND(0.5) → 0.7814688
0.7814688 → NOT → 0.2185312
0.2185312 → AND(0.5) → 0.4370624
0.4370624 → AND(0.5) → 0.8741248
0.8741248 → NOT → 0.1258752
0.1258752 → AND(0.5) → 0.2517504
0.2517504 → AND(0.5) → 0.5035008
0.5035008 → NOT → 0.4964992
0.4964992 → AND(0.5) → 0.9929984
0.9929984 → NOT → 0.0070016
0.0070016 → AND(0.5) → 0.0140032
0.0140032 → AND(0.5) → 0.0280064
0.0280064 → AND(0.5) → 0.0560128
0.0560128 → AND(0.5) → 0.1120256
```

Terminal value reached: 0.1120256

iii. Target = 0.5555555

Cascade steps (reverse reading gives circuit):

```
0.5555555 → NOT → 0.4444445
0.4444445 → AND(0.5) → 0.8888890
0.8888890 → NOT → 0.1111110
0.1111110 → AND(0.5) → 0.2222220
0.2222220 → AND(0.5) → 0.4444440
0.4444440 → AND(0.5) → 0.8888880
0.8888880 → NOT → 0.1111120
0.1111120 → AND(0.5) → 0.2222240
0.2222240 → AND(0.5) → 0.4444480
0.4444480 → AND(0.5) → 0.8888960
0.8888960 → NOT → 0.1111040
0.1111040 → AND(0.5) → 0.2222080
0.2222080 → AND(0.5) → 0.4444160
0.4444160 → AND(0.5) → 0.8888320
0.8888320 → NOT → 0.1111680
0.1111680 → AND(0.5) → 0.2223360
0.2223360 → AND(0.5) → 0.4446720
0.4446720 → AND(0.5) → 0.8893440
0.8893440 → NOT → 0.1106560
0.1106560 → AND(0.5) → 0.2213120
```

Terminal value reached: 0.2213120

CLEAN CIRCUIT CONSTRUCTIONS:

Searching for exact or near circuits using Fraction arithmetic...

i. 0.8881188: Closest exact = 0.8800000 (diff=8.12e-03): NOT(AND(NOT(0.4),AND(0.4,0.5)))

ii. 0.2119209: Closest exact = 0.2100000 (diff=1.92e-03): AND(AND(0.5,0.5),NOT(AND(0.4,0.4)))

iii. 0.5555555: Closest exact = 0.5625000 (diff=6.94e-03): AND(NOT(AND(0.5,0.5)),NOT(AND(0.5,0.5)))

Part (b): Targets from $S = \{0.5\}$ — Binary Probabilities

With only $P=0.5$ available, any binary fraction $0.b_1b_2\dots b_n$ can be synthesized exactly. The algorithm scans bits from LSB (b_n) to MSB (b_1), starting with $p_0 = 0$:

- Bit = 1: $p_{\text{new}} = 0.5 + 0.5 \times p_{\text{prev}} = \text{NOT}(\text{AND}(0.5, \text{NOT}(p_{\text{prev}})))$
- Bit = 0: $p_{\text{new}} = 0.5 \times p_{\text{prev}} = \text{AND}(0.5, p_{\text{prev}})$

This works because each step is equivalent to pre-pending the current bit to the binary fraction being accumulated.

i. $0.1011111_2 = 95/128 = 0.7421875$

Bit position	Bit	Operation	Formula	Running p
b_7 (LSB)	1	$\text{NOT}(\text{AND}(0.5, \text{NOT}(0)))$	$0.5 + 0.5 \times 0$	0.500000
b_6	1	$\text{NOT}(\text{AND}(0.5, \text{NOT}(0.5)))$	$0.5 + 0.5 \times 0.5$	0.750000
b_5	1	$\text{NOT}(\text{AND}(0.5, \text{NOT}(0.75)))$	$0.5 + 0.5 \times 0.75$	0.875000
b_4	1	$\text{NOT}(\text{AND}(0.5, \text{NOT}(0.875)))$	$0.5 + 0.5 \times 0.875$	0.937500
b_3	1	$\text{NOT}(\text{AND}(0.5, \text{NOT}(0.9375)))$	$0.5 + 0.5 \times 0.9375$	0.968750
b_2	0	$\text{AND}(0.5, 0.96875)$	0.5×0.96875	0.484375
b_1 (MSB)	1	$\text{NOT}(\text{AND}(0.5, \text{NOT}(0.484375)))$	$0.5 + 0.5 \times 0.484375$	0.742188 ✓

Result: $0.742188 = 95/128$ ✓

ii. $0.1101111_2 = 111/128 = 0.8671875$

Bit position	Bit	Operation	Formula	Running p
b_7 (LSB)	1	$\text{NOT}(\text{AND}(0.5, \text{NOT}(0)))$	$0.5 + 0.5 \times 0$	0.500000
b_6	1	$\text{NOT}(\text{AND}(0.5, \text{NOT}(0.5)))$	$0.5 + 0.5 \times 0.5$	0.750000
b_5	1	$\text{NOT}(\text{AND}(0.5, \text{NOT}(0.75)))$	$0.5 + 0.5 \times 0.75$	0.875000
b_4	1	$\text{NOT}(\text{AND}(0.5, \text{NOT}(0.875)))$	$0.5 + 0.5 \times 0.875$	0.937500
b_3	0	$\text{AND}(0.5, 0.9375)$	0.5×0.9375	0.468750

Bit position	Bit	Operation	Formula	Running p
b ₂	1	NOT(AND(0.5, NOT(0.46875)))	$0.5 + 0.5 \times 0.46875$	0.734375
b ₁ (MSB)	1	NOT(AND(0.5, NOT(0.734375)))	$0.5 + 0.5 \times 0.734375$	0.867188 ✓

Result: 0.867188 = 111/128 ✓

iii. 0.1010111₂ = 87/128 = 0.6796875

Bit position	Bit	Operation	Formula	Running p
b ₇ (LSB)	1	NOT(AND(0.5, NOT(0)))	$0.5 + 0.5 \times 0$	0.500000
b ₆	1	NOT(AND(0.5, NOT(0.5)))	$0.5 + 0.5 \times 0.5$	0.750000
b ₅	1	NOT(AND(0.5, NOT(0.75)))	$0.5 + 0.5 \times 0.75$	0.875000
b ₄	0	AND(0.5, 0.875)	0.5×0.875	0.437500
b ₃	1	NOT(AND(0.5, NOT(0.4375)))	$0.5 + 0.5 \times 0.4375$	0.718750
b ₂	0	AND(0.5, 0.71875)	0.5×0.71875	0.359375
b ₁ (MSB)	1	NOT(AND(0.5, NOT(0.359375)))	$0.5 + 0.5 \times 0.359375$	0.679688 ✓

Result: 0.679688 = 87/128 ✓

Summary of Part (b) — all results exact:

Case	Binary	Fraction	Decimal	Circuit output
i	0.1011111 ₂	95/128	0.7421875	0.7421875 ✓
ii	0.1101111 ₂	111/128	0.8671875	0.8671875 ✓
iii	0.1010111 ₂	87/128	0.6796875	0.6796875 ✓

```
=====
PROBLEM 2(b): Targets from S = {0.5}
=====
```

i. 0.1011111₂

Binary: 0.1011111

Decimal value: 0.7421875 = 95/128

Circuit construction (right-to-left, LSB first):

b₇ = 1: 0.000000 → bit=1: NOT(AND(0.5, NOT(.))) → 0.500000

b₆ = 1: 0.500000 → bit=1: NOT(AND(0.5, NOT(.))) → 0.750000

b₅ = 1: 0.750000 → bit=1: NOT(AND(0.5, NOT(.))) → 0.875000

b₄ = 1: 0.875000 → bit=1: NOT(AND(0.5, NOT(.))) → 0.937500

b₃ = 1: 0.937500 → bit=1: NOT(AND(0.5, NOT(.))) → 0.968750

b₂ = 0: 0.968750 → bit=0: AND(0.5, .) → 0.484375

b₁ = 1: 0.484375 → bit=1: NOT(AND(0.5, NOT(.))) → 0.742188

Final: p = 0.7421875

Circuit: NOT(AND(0.5, NOT(AND(0.5, NOT(AND(0.5, NOT(NOT(AND(0.5, NOT(NOT(AND(0.5, NOT(NOT(AND(0.5, NOT(0)))))))))))))))

ii. 0.1101111₂

Binary: 0.1101111

Decimal value: 0.8671875 = 111/128

Circuit construction (right-to-left, LSB first):

b₇ = 1: 0.000000 → bit=1: NOT(AND(0.5, NOT(.))) → 0.500000

b₆ = 1: 0.500000 → bit=1: NOT(AND(0.5, NOT(.))) → 0.750000

b₅ = 1: 0.750000 → bit=1: NOT(AND(0.5, NOT(.))) → 0.875000

b₄ = 1: 0.875000 → bit=1: NOT(AND(0.5, NOT(.))) → 0.937500

b₃ = 0: 0.937500 → bit=0: AND(0.5, .) → 0.468750

b₂ = 1: 0.468750 → bit=1: NOT(AND(0.5, NOT(.))) → 0.734375

b₁ = 1: 0.734375 → bit=1: NOT(AND(0.5, NOT(.))) → 0.867188

Final: p = 0.8671875

Circuit: NOT(AND(0.5, NOT(NOT(AND(0.5, NOT(AND(0.5, NOT(AND(0.5, NOT(NOT(AND(0.5, NOT(NOT(AND(0.5, NOT(0)))))))))))))))

iii. 0.1010111₂

Binary: 0.1010111

Decimal value: 0.6796875 = 87/128

Circuit construction (right-to-left, LSB first):

b₇ = 1: 0.000000 → bit=1: NOT(AND(0.5, NOT(.))) → 0.500000

b₆ = 1: 0.500000 → bit=1: NOT(AND(0.5, NOT(.))) → 0.750000

b₅ = 1: 0.750000 → bit=1: NOT(AND(0.5, NOT(.))) → 0.875000

b₄ = 0: 0.875000 → bit=0: AND(0.5, .) → 0.437500

b₃ = 1: 0.437500 → bit=1: NOT(AND(0.5, NOT(.))) → 0.718750

b₂ = 0: 0.718750 → bit=0: AND(0.5, .) → 0.359375

b₁ = 1: 0.359375 → bit=1: NOT(AND(0.5, NOT(.))) → 0.679688

Final: p = 0.6796875

Circuit: NOT(AND(0.5, NOT(AND(0.5, NOT(AND(0.5, NOT(AND(0.5, NOT(AND(0.5, NOT(NOT(AND(0.5, NOT(NOT(AND(0.5, NOT(0)))))))))))))))

```
=====
Summary of binary probability circuits
=====
```

i. 0.1011111₂ = 95/128 = 0.7421875, circuit output = 0.7421875

ii. 0.1101111₂ = 111/128 = 0.8671875, circuit output = 0.8671875

iii. 0.1010111₂ = 87/128 = 0.6796875, circuit output = 0.6796875